NATIONAL ECONOMICS UNIVERSITY
COLLEGE OF TECHNOLOGY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

MIDTERM PROJECT REPORT

INTRODUCTION TO DATABASE (TOKT11120.AI67A)

TOPIC: HAIR SALON MANAGEMENT

Instructor: Mr. Tran Duc Minh

Students:	Ung Hoang Anh	11251042
	Nguyen Phuc Long	11254989
	Nguyen Doan Hoang Linh	11254505

Hanoi, October 2026

TABLE OF CONTENTS

1 INTRODUCTION **1**
1.1 Background 1
1.2 Project goals 1
1.3 Development environment 1
2 REQUIREMENTS ANALYSIS **2**
2.1 Business context 2
2.2 Functional requirements 2
2.3 Business rules 2
2.4 Data requirements and sample scenarios 3
3 DATABASE DESIGN **4**
3.1 Entity identification 4
3.2 Relationships and cardinalities 4
3.3 Relational schema 5
3.4 Table structures and attributes 6
4 DATABASE IMPLEMENTATION AND TESTING **8**
4.1 Database creation 8
4.2 SQL file organization and execution order 8
4.3 Sample data overview 9
4.4 Data integrity validation 9
4.5 Sample business cases 11
5 SQL QUERY IMPLEMENTATION **11**
5.1 Query 1 – Completed appointments in January 2025 11
5.2 Query 2 – Appointment count by service team in Q1 2025 12
5.3 Query 3 – Customers with appointments without service records 13
5.4 Query 4 – Average completed service cost by team 13
5.5 Query 5 – Top three stylists by completed appointments in 2025 14
5.6 Query 6 – Total payments received from each customer in 2025 15
5.7 Query 7 – Customers with no payment records 16
5.8 Query 8 – Customers ranked by total confirmed service cost 16
5.9 Query 9 – Cancellation rate by stylist in 2025 17
5.10 Query 10 – Customers with more than three completed appointments in 2025 18
5.11 Summary of SQL techniques 18
6 DATABASE VIEW IMPLEMENTATION **19**
6.1 View `v_detail` – Appointment details 19
6.2 View `v_group_summary` – Service-team summary 19
6.3 View `v_debt` – Customer debt tracking 20
6.4 View usage 21

LIST OF TABLES

1.1 Development environment specification 1
2.1 Functional requirements and their database support 2
2.2 Required scenarios and their representation in the sample data 3
3.1 Entities and their corresponding tables 4
3.2 Relationships and cardinalities 4
3.3 Primary keys and foreign keys of the relational schema 5
3.4 Table structures and attributes 6
4.1 SQL file organization and execution order 8
4.2 Number of sample records per table 9
4.3 Data integrity validation checks 10
4.4 Sample business cases in the dataset 11
5.1 Expected matching appointment records based on the supplied seed data . 12
5.2 Expected counts based on the supplied sample data 13
5.3 Expected team-level service-cost statistics based on the supplied seed data 14
5.4 Expected cancellation rates based on the supplied sample data 17
5.5 SQL techniques used in the project 18
6.1 Expected output of `v_group_summary` based on all sample records 20
6.2 Selected expected debt records based on the supplied sample data (VND) 21

LIST OF FIGURES

3.1 Logical relationship overview (ERD) of the salon database 5

1 INTRODUCTION

1.1 Background

Hair salons handle several connected business activities every day. These activities include receiving customer appointments, assigning stylists, recording completed services, confirming service fees, and collecting payments. When information is stored in separate files or managed manually, it can become difficult to retrieve a customer's service history, monitor the workload of each stylist, summarize the performance of service teams, or identify outstanding payments.

A relational database can provide a structured way to store and connect this information. By organizing business data into related tables and enforcing integrity constraints, the salon can reduce inconsistencies and support reliable reporting. This project therefore focuses on building a database that represents the essential workflow of a hair salon.

1.2 Project goals

The main objective is to design and implement a relational database for the management of a hair salon. The specific objectives are:

- Identify the main entities involved in salon operations and define their relationships.
- Create six relational tables with suitable primary keys, foreign keys, data types, and integrity constraints.
- Insert sample data that covers the required business scenarios and different time periods.
- Use SQL operations such as `JOIN`, `LEFT JOIN`, `GROUP BY`, `HAVING`, subqueries, and aggregate functions to answer business questions.
- Create three database views for appointment details, service-team performance, and customer debt tracking.

1.3 Development environment

Table 1.1. Development environment specification

Component	Specification
Database Management System	MySQL 8.0.16 or later
Storage Engine	InnoDB
Character Set	`utf8mb4`
Database Name	`salon_mini`
Implementation Language	SQL

2 REQUIREMENTS ANALYSIS

2.1 Business context

The salon receives appointments from customers and assigns each appointment to one stylist. Stylists belong to service teams, such as cutting and styling, hair coloring, perming and straightening, hair restoration, or hair washing and relaxation. When a service is completed, the salon records a service summary and the total confirmed cost. Payments are recorded as actual collection events, and a single appointment may be paid in multiple transactions.

2.2 Functional requirements

Table 2.1. Functional requirements and their database support

No.	Requirement	Database support
FR-01	Store service teams.	`service_teams`
FR-02	Store stylists and their team assignments.	`stylists`
FR-03	Store customer information.	`customers`
FR-04	Record appointments, assigned stylists, dates, and statuses.	`appointments`
FR-05	Record completed services and confirmed costs.	`service_records`
FR-06	Record one or more actual payments for an appointment.	`payments`
FR-07	Retrieve appointment and customer information.	SQL queries and `v_detail`
FR-08	Summarize activity by service team.	SQL queries and `v_group_summary`
FR-09	Calculate customer payment totals and outstanding debt.	SQL queries and `v_debt`

2.3 Business rules

- A service team may have multiple stylists, while each stylist belongs to exactly one service team.
- Each appointment belongs to exactly one customer and is assigned to exactly one main stylist.
- The appointment status must be one of Pending, Completed, or Cancelled.
- Each appointment may have at most one service record. Every Completed appointment must have one service record, while Pending and Cancelled appointments must

not have a service record.

- The confirmed service cost must be greater than or equal to zero.
- Each payment row represents one actual collection event. Its amount must be greater than zero, and its method must be Cash, BankTransfer, or Card.
- An appointment may have multiple payment records, but the total amount collected must not exceed its confirmed service cost.
- Payments are recorded only after a service cost has been confirmed. Advance payments and refunds are outside the scope of the system.
- When service costs and payments are combined for reporting, service costs must not be duplicated because of multiple payment rows.
- Reports specified by year use 2025. The first quarter of 2025 covers January 1 through March 31. Queries without a specified period use all available data.

2.4 Data requirements and sample scenarios

The sample dataset must meet the minimum row counts and include different business situations. The implementation provides five service teams, seven stylists, twelve customers, twenty-one appointments, fifteen service records, and fifteen payment records.

Table 2.2. Required scenarios and their representation in the sample data

Required scenario	How it is represented in the sample data
All appointment statuses	Pending, Completed, and Cancelled appointments are included.
More than three Completed appointments in 2025	Customer 1 has four Completed appointments in 2025.
Appointment history without payment	Customer 8 has a Completed appointment with no payment record.
Customer with no appointment history	Customer 11 has no appointment.
Multiple payments for one appointment	Appointments 4 and 11 are paid in multiple transactions.
Outstanding debt	Appointment 7 is partially paid; appointment 10 is unpaid.
Service team with no Q1 2025 appointment	Service team 5 has no appointment in the first quarter of 2025.
Stylist with no appointment in 2025	Stylist 7 has only a 2024 appointment; stylist 6 has no appointment.

3 DATABASE DESIGN

3.1 Entity identification

Based on the business requirements, six entities are identified. Each entity is implemented as a relational table.

Table 3.1. Entities and their corresponding tables

Entity / Table	Description
`service_teams`	Stores the salon's specialist service teams.
`stylists`	Stores stylist information and the team to which each stylist belongs.
`customers`	Stores customer contact information and preference notes.
`appointments`	Stores appointment dates, statuses, customers, and assigned stylists.
`service_records`	Stores the service summary, product note, and confirmed cost for a completed appointment.
`payments`	Stores each actual payment transaction made for an appointment.

3.2 Relationships and cardinalities

Table 3.2. Relationships and cardinalities

Relationship	Cardinality	Explanation
`service_teams` – `stylists`	1 : N	One team can contain many stylists; each stylist belongs to one team.
`customers` – `appointments`	1 : N	One customer can make many appointments; each appointment belongs to one customer.
`stylists` – `appointments`	1 : N	One stylist can be assigned to many appointments; each appointment has one main stylist.
`appointments` – `service_records`	1 : 0..1	An appointment has at most one service record; a Completed appointment must have one.
`appointments` – `payments`	1 : 0..N	An appointment may have no payment yet or multiple payment transactions.

Figure 3.1 shows the six entities, their primary keys, foreign keys, and relationship cardinalities.

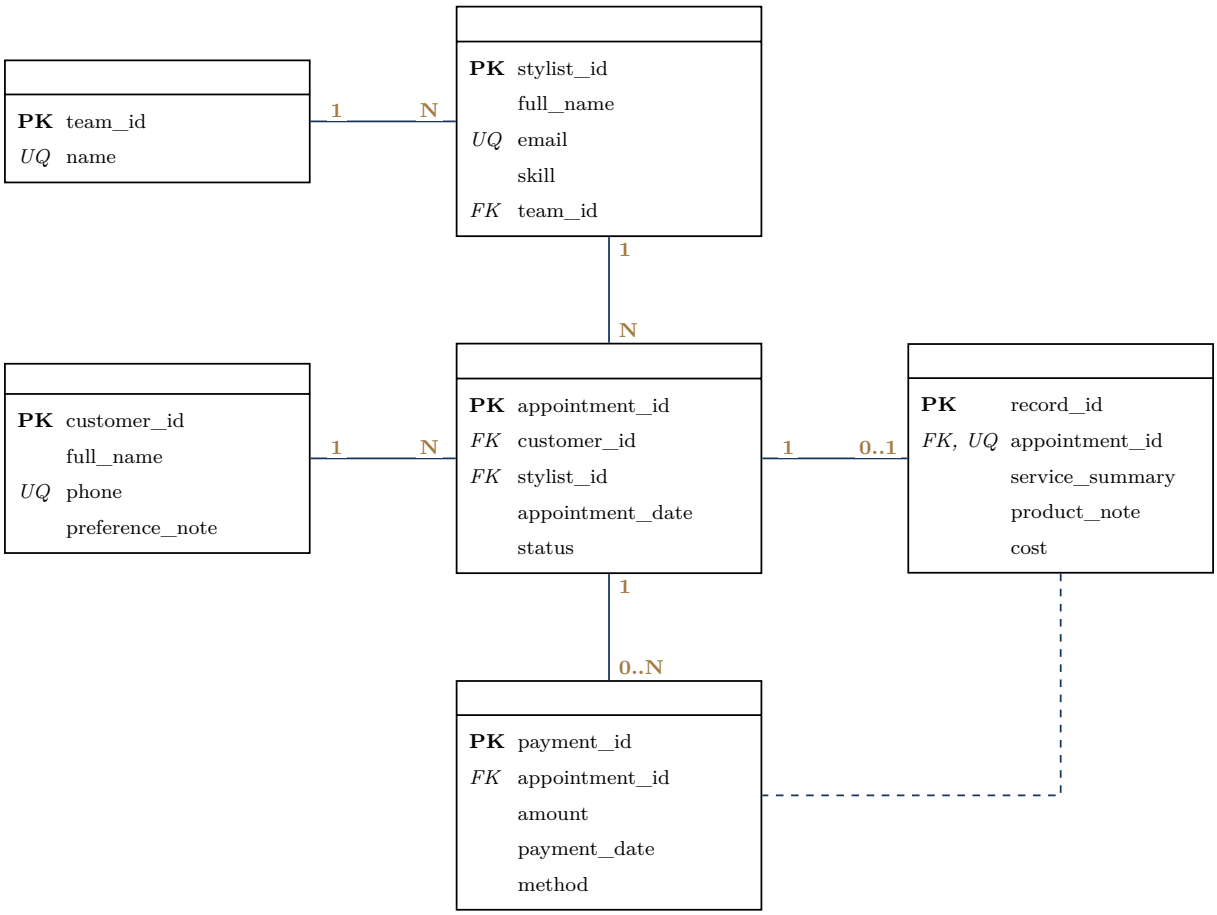


Figure 3.1. Logical relationship overview (ERD) of the salon database. PK: primary key; FK: foreign key; UQ: unique. The dashed line marks the second foreign key of `payments.appointment_id`, which also references `service_records`.

3.3 Relational schema

Table 3.3. Primary keys and foreign keys of the relational schema

Table	Primary Key	Foreign Keys
`service_teams`	`team_id`	None
`stylists`	`stylist_id`	`team_id` → `service_teams.team_id`
`customers`	`customer_id`	None
`appointments`	`appointment_id`	`customer_id` → `customers.customer_id`; `stylist_id` → `stylists.stylist_id`
`service_records`	`record_id`	`appointment_id` → `appointments.appointment_id`
`payments`	`payment_id`	`appointment_id` → `appointments.appointment_id`; `appointment_id` → `service_records.appointment_id`

The payments table contains two foreign-key constraints on appointment_id. One references appointments to ensure that the appointment exists. The other references the unique column appointment_id of service_records, to ensure that a payment can only be recorded for an appointment that already has a confirmed service record.

3.4 Table structures and attributes

Table 3.4. Table structures and attributes

Table	Attribute	Data type	Constraint / Meaning
service_teams	team_id	INT	PK, AUTO_INCREMENT, NOT NULL
	name	VARCHAR(100)	NOT NULL, UNIQUE, non-empty
stylists	stylist_id	INT	PK, AUTO_INCREMENT
	full_name	VARCHAR(100)	NOT NULL, non-empty
	email	VARCHAR(150)	NOT NULL, UNIQUE, basic format CHECK
	skill	VARCHAR(150)	Optional
	team_id	INT	NOT NULL, FK
customers	customer_id	INT	PK, AUTO_INCREMENT
	full_name	VARCHAR(100)	NOT NULL, non-empty
	phone	VARCHAR(15)	NOT NULL, UNIQUE, format CHECK
	preference_note	VARCHAR(255)	Optional
appointments	appointment_id	INT	PK, AUTO_INCREMENT
	customer_id	INT	NOT NULL, FK
	stylist_id	INT	NOT NULL, FK
	appointment_date	DATETIME	NOT NULL
	status	ENUM	Pending / Completed / Cancelled
service_records	record_id	INT	PK, AUTO_INCREMENT
	appointment_id	INT	NOT NULL, UNIQUE, FK
	service_summary	VARCHAR(255)	NOT NULL, non-empty
	product_note	VARCHAR(255)	Optional
	cost	DECIMAL(12,2)	NOT NULL, CHECK cost ≥ 0
payments	payment_id	INT	PK, AUTO_INCREMENT

Continued on the next page

Table 3.4 (continued)

Table	Attribute	Data type	Constraint / Meaning
	`appointment_id`	`INT`	NOT NULL, FK to `appointments` and `service_records`
	`amount`	`DECIMAL(12,2)`	NOT NULL, CHECK amount > 0
	`payment_date`	`DATETIME`	NOT NULL
	`method`	`ENUM`	Cash / BankTransfer / Card

4 DATABASE IMPLEMENTATION AND TESTING

4.1 Database creation

The database is named `salon_mini` and uses the `utf8mb4` character set with `utf8mb4_unicode_ci` collation. The `schema.sql` file creates the database and then defines the six tables in dependency order so that referenced parent tables exist before their foreign keys are declared.

```
CREATE DATABASE salon_mini
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
USE salon_mini;
```

4.2 SQL file organization and execution order

Table 4.1. SQL file organization and execution order

File	Purpose	Execution order
`schema.sql`	Creates the database, tables, keys, indexes, and constraints.	1
`seed.sql`	Inserts sample data and runs integrity/business-rule checks.	2
`queries.sql`	Contains the ten required data retrieval queries.	3
`views.sql`	Creates three views and selects their contents for inspection.	4

The files must be executed in the order shown above on a fresh database. The `schema.sql` script begins by dropping an existing `salon_mini` database, so it should only be run when recreating the project database is intended.

4.3 Sample data overview

Table 4.2. Number of sample records per table

Table	Number of records	Minimum required
`service_teams`	5	5
`stylists`	7	6
`customers`	12	10
`appointments`	21	12
`service_records`	15	10
`payments`	15	10
Total	**75**	—

The sample data includes appointments in December 2024, throughout 2025, and in 2026. In 2025, the data includes appointments in January, February, March, and later months. This makes it possible to test both period-specific queries and views that consider all available records.

4.4 Data integrity validation

After inserting the sample records, `seed.sql` executes validation queries. These checks are designed to return the number of invalid rows. A result of zero indicates that no violation was found for the tested condition.

Table 4.3. Data integrity validation checks

Validation area	What is checked	Expected condition
Foreign-key integrity	Child records must reference existing parent records.	Zero orphan records.
Appointment/service consistency	Completed appointments have service records; non-Completed appointments do not.	Zero inconsistent records.
Service-record uniqueness	An appointment has no more than one service record.	No duplicate `appointment_id`.
Payment eligibility	Payments belong to Completed appointments with confirmed service records.	Zero invalid payments.
Payment ceiling	Total collected for an appointment does not exceed its confirmed cost.	Zero overpaid appointments.
Payment chronology	Payment date is not earlier than the appointment date.	No chronology violations.

The script also checks the presence of required sample scenarios, such as multiple payments for one appointment, outstanding debt, a customer without payment history, a team without first-quarter activity, and a stylist without activity in 2025.

4.5 Sample business cases

Table 4.4. Sample business cases in the dataset

Case	Example from the dataset	Business interpretation
Multiple payments	Appointment 4: 500,000 + 700,000 VND; appointment 11: 1,000,000 + 500,000 VND.	A service transaction can be settled through multiple collections.
Partial payment	Appointment 7: cost 900,000 VND; paid 400,000 VND.	Remaining debt is 500,000 VND.
Unpaid service	Appointment 10: cost 800,000 VND; no payment.	The full 800,000 VND remains outstanding.
Free service	Appointment 18: cost 0 VND.	A completed service can have zero cost and does not require a payment.
Customer with no appointment	Customer 11.	The customer should still appear in customer-level reports.
Customer with more than three completed appointments	Customer 1: appointments 3, 8, 11, and 14 in 2025.	Supports the `HAVING` query for frequent customers.

5 SQL QUERY IMPLEMENTATION

The `queries.sql` file contains ten independent queries. The queries are designed to answer the reporting questions specified in the assignment. Date filters use half-open intervals, such as greater than or equal to the first day of a period and less than the first day of the next period. This includes every time value on the final day without relying on a time component of 23:59:59.

5.1 Query 1 – Completed appointments in January 2025

> **Purpose:** Retrieve all Completed appointments in January 2025, including the appointment date, customer, and assigned stylist.

```
SELECT a.appointment_id,
       a.appointment_date,
       a.status,
       c.customer_id,
       c.full_name AS customer_name,
       s.stylist_id,
       s.full_name AS stylist_name
```

```
FROM appointments a
JOIN customers c ON c.customer_id = a.customer_id
JOIN stylists s ON s.stylist_id = a.stylist_id
WHERE a.status = 'Completed'
  AND a.appointment_date >= '2025-01-01'
  AND a.appointment_date <  '2025-02-01'
ORDER BY a.appointment_date, a.appointment_id;
```

The query uses `INNER JOIN` because each appointment must be associated with an existing customer and stylist. The date range includes appointments from January 1 up to, but not including, February 1. Based on the sample data, the matching appointment IDs are 3, 4, 6, and 7.

Table 5.1. Expected matching appointment records based on the supplied seed data

Appointment ID	Date	Customer ID	Stylist ID	Status
3	2025-01-05	1	1	Completed
4	2025-01-10	3	3	Completed
6	2025-01-20	7	1	Completed
7	2025-01-28	5	4	Completed

5.2 Query 2 – Appointment count by service team in Q1 2025

> **Purpose:** Count all appointments, regardless of status, for each service team during the first quarter of 2025. Teams with no appointments must also be displayed.

```
SELECT t.team_id,
       t.name AS team_name,
       COUNT(a.appointment_id) AS total_appointments_q1_2025
FROM service_teams t
LEFT JOIN stylists s
       ON s.team_id = t.team_id
LEFT JOIN appointments a
       ON a.stylist_id = s.stylist_id
      AND a.appointment_date >= '2025-01-01'
      AND a.appointment_date <  '2025-04-01'
GROUP BY t.team_id, t.name
ORDER BY t.team_id;
```

`LEFT JOIN` preserves service teams even when no appointment matches the date range. The date condition is placed in the `ON` clause rather than the `WHERE` clause; placing it in `WHERE` would remove rows with no matching appointment. `COUNT(a.appointment_id)` returns zero for a team without appointments.

Table 5.2. Expected counts based on the supplied sample data

Team ID	Service team	Q1 2025 appointments
1	Cutting and Styling	5
2	Hair Coloring	3
3	Perming and Straightening	3
4	Hair Restoration	0
5	Washing and Relaxation	0

5.3 Query 3 – Customers with appointments without service records

> **Purpose:** Find customers who have at least one appointment without a service record, excluding Cancelled appointments. Each customer must appear only once.

```sql
SELECT c.customer_id,
       c.full_name AS customer_name,
       c.phone
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM appointments a
    LEFT JOIN service_records sr
           ON sr.appointment_id = a.appointment_id
    WHERE a.customer_id = c.customer_id
      AND a.status <> 'Cancelled'
      AND sr.record_id IS NULL
)
ORDER BY c.customer_id;
```

The correlated `EXISTS` subquery checks whether at least one qualifying appointment exists for each customer. `LEFT JOIN` identifies appointments without a matching service record, while the status condition excludes cancelled appointments. `EXISTS` returns each customer once even if multiple qualifying appointments exist. In the sample data, customers 5 and 12 meet this condition because of Pending appointments 20 and 21 respectively.

5.4 Query 4 – Average completed service cost by team

> **Purpose:** Calculate the number of completed transactions, total service cost, and average service cost for each team across all available dates. Teams without completed service costs are excluded.

```sql
SELECT t.team_id,
       t.name AS team_name,
       COUNT(sr.record_id) AS completed_count,
```

```
        SUM(sr.cost) AS total_cost,
        ROUND(AVG(sr.cost), 2) AS avg_cost_per_completed
FROM service_teams t
JOIN stylists s
     ON s.team_id = t.team_id
JOIN appointments a
     ON a.stylist_id = s.stylist_id
    AND a.status = 'Completed'
JOIN service_records sr
     ON sr.appointment_id = a.appointment_id
GROUP BY t.team_id, t.name
HAVING COUNT(sr.record_id) > 0
ORDER BY t.team_id;
```

The query joins service teams to stylists, completed appointments, and service records. `AVG` calculates the average cost per completed transaction, while `ROUND` formats the result to two decimal places. Payments are intentionally excluded from this query so that multiple payment rows cannot duplicate a service cost.

Table 5.3. Expected team-level service-cost statistics based on the supplied seed data

Team	Completed records	Total cost (VND)	Average cost (VND)
Cutting and Styling	7	1,350,000	192,857.14
Hair Coloring	3	3,350,000	1,116,666.67
Perming and Straightening	3	2,050,000	683,333.33
Hair Restoration	2	700,000	350,000.00

5.5 Query 5 – Top three stylists by completed appointments in 2025

> **Purpose:** Identify the three stylists with the highest number of Completed appointments in 2025. If two stylists have the same count, the smaller `stylist_id` is ranked first.

```
SELECT s.stylist_id,
       s.full_name AS stylist_name,
       t.team_id,
       t.name AS team_name,
       COUNT(a.appointment_id) AS completed_count_2025
FROM stylists s
JOIN service_teams t ON t.team_id = s.team_id
JOIN appointments a ON a.stylist_id = s.stylist_id
WHERE a.status = 'Completed'
  AND a.appointment_date >= '2025-01-01'
```

```
  AND a.appointment_date <  '2026-01-01'
GROUP BY s.stylist_id, s.full_name, t.team_id, t.name
ORDER BY completed_count_2025 DESC, s.stylist_id ASC
LIMIT 3;
```

The `INNER JOIN` includes only stylists who have at least one qualifying Completed appointment. `GROUP BY` counts each stylist's completed appointments. `ORDER BY` applies the requested tie-break rule, and `LIMIT 3` returns only the top three rows. Based on the sample data, stylist IDs 1 and 4 have three Completed appointments each. Stylist ID 2 ranks next with two Completed appointments; it is selected as the third row because the query breaks ties by ascending `stylist_id`.

5.6 Query 6 – Total payments received from each customer in 2025

> **Purpose:** Calculate the amount each customer actually paid during 2025, based on `payment_date`. Customers without payments during the year must still appear with a total of zero.

```
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(pay.total_paid, 0) AS total_paid_2025
FROM customers c
LEFT JOIN (
    SELECT a.customer_id,
           SUM(p.amount) AS total_paid
    FROM payments p
    JOIN appointments a
         ON a.appointment_id = p.appointment_id
    WHERE p.payment_date >= '2025-01-01'
      AND p.payment_date <  '2026-01-01'
    GROUP BY a.customer_id
) pay ON pay.customer_id = c.customer_id
ORDER BY c.customer_id;
```

The subquery first aggregates payments by customer and filters them using `payment_date`. The outer query starts from customers and uses `LEFT JOIN` to retain customers with no payment in 2025. `COALESCE` changes a missing aggregate value to zero. The query measures cash actually received during the year, not the service date or appointment date.

For example, customer 1 has payments totaling 2,080,000 VND during 2025. Customer 3 has 1,900,000 VND, while customers with no 2025 payments are displayed with zero.

5.7 Query 7 – Customers with no payment records

> **Purpose:** Find customers who have never made a payment in the entire dataset, including customers who have never made an appointment.

```sql
SELECT c.customer_id,
       c.full_name AS customer_name,
       c.phone
FROM customers c
WHERE NOT EXISTS (
    SELECT 1
    FROM appointments a
    JOIN payments p
         ON p.appointment_id = a.appointment_id
    WHERE a.customer_id = c.customer_id
)
ORDER BY c.customer_id;
```

`NOT EXISTS` returns a customer only when the subquery cannot find any appointment associated with a payment record. This approach includes both customers with unpaid appointments and customers with no appointment history. In the sample data, customer IDs 8, 9, 10, 11, and 12 have no payment records.

5.8 Query 8 – Customers ranked by total confirmed service cost

> **Purpose:** List all customers in descending order of their total confirmed service costs, regardless of how much they have paid. Customers without confirmed service costs must also be included.

```sql
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(SUM(sr.cost), 0) AS total_confirmed_cost
FROM customers c
LEFT JOIN appointments a
       ON a.customer_id = c.customer_id
      AND a.status = 'Completed'
LEFT JOIN service_records sr
       ON sr.appointment_id = a.appointment_id
GROUP BY c.customer_id, c.full_name
ORDER BY total_confirmed_cost DESC, c.customer_id ASC;
```

The query uses `LEFT JOIN` to preserve every customer. It sums service-record costs from Completed appointments and does not join the `payments` table. This is important because a completed appointment may have several payment rows; joining payments directly could multiply the service cost and produce an incorrect total.

5.9 Query 9 – Cancellation rate by stylist in 2025

> **Purpose:** Calculate the cancellation rate for each stylist during 2025. The rate is the number of Cancelled appointments divided by all appointments assigned to the stylist in 2025, multiplied by 100. The result is rounded to two decimal places.

```sql
SELECT s.stylist_id,
       s.full_name AS stylist_name,
       COUNT(a.appointment_id) AS total_appointments_2025,
       SUM(CASE WHEN a.status = 'Cancelled' THEN 1 ELSE 0 END)
           AS cancelled_count_2025,
       ROUND(
           100.0 * SUM(CASE WHEN a.status = 'Cancelled' THEN 1 ELSE
    0 END)
           / NULLIF(COUNT(a.appointment_id), 0),
           2
       ) AS cancel_rate_percent
FROM stylists s
LEFT JOIN appointments a
       ON a.stylist_id = s.stylist_id
      AND a.appointment_date >= '2025-01-01'
      AND a.appointment_date <  '2026-01-01'
GROUP BY s.stylist_id, s.full_name
ORDER BY s.stylist_id;
```

`CASE` counts cancelled appointments, while `COUNT` counts all appointments in the selected period. `NULLIF` changes a zero denominator to `NULL`, preventing division by zero. `LEFT JOIN` ensures that stylists without appointments in 2025 remain in the result; their cancellation rate is `NULL`.

Table 5.4. Expected cancellation rates based on the supplied sample data

Stylist ID	Appointments	Cancelled	Cancellation rate
1	4	1	25.00%
2	3	1	33.33%
3	3	1	33.33%
4	3	0	0.00%
5	3	1	33.33%
6	0	0	NULL
7	0	0	NULL

5.10 Query 10 – Customers with more than three completed appointments in 2025

> **Purpose:** Identify customers who had more than three Completed appointments in 2025 and display the number of qualifying appointments.

```sql
SELECT c.customer_id,
       c.full_name AS customer_name,
       COUNT(a.appointment_id) AS completed_count_2025
FROM customers c
JOIN appointments a
     ON a.customer_id = c.customer_id
WHERE a.status = 'Completed'
  AND a.appointment_date >= '2025-01-01'
  AND a.appointment_date <  '2026-01-01'
GROUP BY c.customer_id, c.full_name
HAVING COUNT(a.appointment_id) > 3
ORDER BY completed_count_2025 DESC, c.customer_id ASC;
```

`HAVING` filters the grouped results after the appointment count has been calculated. The condition `> 3` ensures that customers with exactly three Completed appointments are not included. In the sample data, customer 1 has four Completed appointments in 2025 and is returned by the query.

5.11 Summary of SQL techniques

Table 5.5. SQL techniques used in the project

SQL technique	Purpose in this project
`INNER JOIN`	Combines records when matching related rows are required.
`LEFT JOIN`	Preserves parent rows when related activity may be missing.
`GROUP BY`	Aggregates appointments, payments, or service costs by entity.
`HAVING`	Filters groups after aggregation, such as customers with more than three appointments.
`EXISTS / NOT EXISTS`	Checks whether related qualifying or disqualifying records exist.
`CASE`	Counts appointments that satisfy a specific status condition.
`COALESCE`	Displays zero instead of `NULL` for missing aggregate values.
`NULLIF`	Prevents division by zero in cancellation-rate calculations.
`SUM / AVG / COUNT`	Calculates totals, averages, and record counts.
`ORDER BY / LIMIT`	Sorts reports and returns a limited number of ranked rows.

6 DATABASE VIEW IMPLEMENTATION

Three views are created in `views.sql`. Unlike queries that explicitly filter by a particular reporting period, these views summarize all available records. Each view is designed around a different management purpose.

6.1 View `v_detail` – Appointment details

The `v_detail` view returns one row per appointment. It includes the appointment ID, appointment date, status, customer information, stylist information, service team, and confirmed service cost. A `LEFT JOIN` is used for `service_records` so that appointments without a service record are still displayed. For such appointments, `service_cost` is `NULL`.

```
CREATE OR REPLACE VIEW v_detail AS
SELECT a.appointment_id,
       a.appointment_date,
       a.status,
       c.customer_id,
       c.full_name AS customer_name,
       s.stylist_id,
       s.full_name AS stylist_name,
       t.team_id,
       t.name AS team_name,
       sr.cost AS service_cost
FROM appointments a
JOIN customers c ON c.customer_id = a.customer_id
JOIN stylists s ON s.stylist_id = a.stylist_id
JOIN service_teams t ON t.team_id = s.team_id
LEFT JOIN service_records sr
       ON sr.appointment_id = a.appointment_id;
```

The view is useful when staff need to inspect an appointment together with the customer, assigned stylist, team, and service fee without manually joining several tables.

6.2 View `v_group_summary` – Service-team summary

The `v_group_summary` view returns one row for every service team. It includes the total number of appointments, the number of Completed appointments, the number of Cancelled appointments, and the total service cost. `LEFT JOIN` is used throughout the relationship path so that teams with no appointments are retained. `COALESCE` ensures that aggregate values for teams without activity are displayed as zero.

```
CREATE OR REPLACE VIEW v_group_summary AS
SELECT t.team_id,
       t.name AS team_name,
       COUNT(a.appointment_id) AS total_appointments,
       COALESCE(SUM(
           CASE WHEN a.status = 'Completed' THEN 1 ELSE 0 END
       ), 0) AS completed_count,
```

```
        COALESCE(SUM(
            CASE WHEN a.status = 'Cancelled' THEN 1 ELSE 0 END
        ), 0) AS cancelled_count,
        COALESCE(SUM(sr.cost), 0) AS total_service_cost
FROM service_teams t
LEFT JOIN stylists s ON s.team_id = t.team_id
LEFT JOIN appointments a ON a.stylist_id = s.stylist_id
LEFT JOIN service_records sr
        ON sr.appointment_id = a.appointment_id
GROUP BY t.team_id, t.name;
```

Based on the sample dataset, the view should show five rows. Service team 5 has no stylists with appointments and therefore has zero appointments, zero completed appointments, zero cancellations, and zero service cost.

Table 6.1. Expected output of `v_group_summary` based on all sample records

Team	Total appointments	Completed	Cancelled	Total cost (VND)
Cutting and Styling	10	7	2	1,350,000
Hair Coloring	4	3	1	3,350,000
Perming and Straightening	4	3	0	2,050,000
Hair Restoration	3	2	1	700,000
Washing and Relaxation	0	0	0	0

The figures above are calculated from all appointment dates in `seed.sql`, not only from 2025. The total appointment count includes Pending, Completed, and Cancelled appointments, while the total service cost is the sum of the confirmed costs recorded for the team. Teams without appointments are retained and display zero values.

6.3 View `v_debt` – Customer debt tracking

The `v_debt` view returns one row per customer and provides three financial measures: total confirmed service cost, total amount paid, and remaining debt. Customers without completed services or payments are also retained in the view, with missing totals converted to zero.

The most important design consideration is avoiding duplicated service costs. If `service_records` were joined directly to `payments` before aggregation, an appointment with multiple payment rows could cause its service cost to appear more than once. To avoid this, the view calculates the total service cost and total payments in two separate subqueries, groups each subquery by customer, and then joins the results to `customers`.

```sql
CREATE OR REPLACE VIEW v_debt AS
SELECT c.customer_id,
       c.full_name AS customer_name,
       COALESCE(cost_sum.total_cost, 0) AS total_confirmed_cost,
       COALESCE(pay_sum.total_paid, 0) AS total_paid,
       COALESCE(cost_sum.total_cost, 0)
         - COALESCE(pay_sum.total_paid, 0) AS remaining_debt
FROM customers c
LEFT JOIN (
    SELECT a.customer_id,
           SUM(sr.cost) AS total_cost
    FROM appointments a
    JOIN service_records sr
         ON sr.appointment_id = a.appointment_id
    WHERE a.status = 'Completed'
    GROUP BY a.customer_id
) cost_sum ON cost_sum.customer_id = c.customer_id
LEFT JOIN (
    SELECT a.customer_id,
           SUM(p.amount) AS total_paid
    FROM appointments a
    JOIN payments p
         ON p.appointment_id = a.appointment_id
    GROUP BY a.customer_id
) pay_sum ON pay_sum.customer_id = c.customer_id;
```

The debt calculation is:

$$\text{Remaining debt} = \text{Total confirmed service cost} - \text{Total amount paid}$$

Based on the sample data, customer 5 has 500,000 VND in remaining debt, while customer 8 has 800,000 VND in remaining debt. Customers whose service costs have been fully paid have a remaining debt of zero.

Table 6.2. Selected expected debt records based on the supplied sample data (VND)

Customer ID	Total confirmed cost	Total paid	Remaining debt
5	900,000	400,000	500,000
8	800,000	0	800,000
1	2,230,000	2,230,000	0
11	0	0	0

6.4 View usage

After creation, the views can be queried like regular tables. The `views.sql` file includes `SELECT` statements to inspect all three views:

```sql
SELECT * FROM v_detail ORDER BY appointment_id;
SELECT * FROM v_group_summary ORDER BY team_id;
SELECT * FROM v_debt ORDER BY customer_id;
```

These views reduce repeated query-writing effort and provide consistent structures for common reporting tasks. They do not replace the underlying tables; instead, they present selected and combined data from those tables.